Chapitre 2 — Représentation informatique

Ce chapitre répond à une question pratique : comment **stocker un graphe en mémoire** pour pouvoir le manipuler dans un programme ? Le PDF présente deux représentations principales (la **matrice d'adjacences** et la **liste d'adjacence**), puis des critères de choix et des types spécialisés. Il est couvert par les pages 16 à 22 du PDF.

Section	Sujet	Pages PDF
2.1	Page de séparation	16
2.2	Matrice d'adjacences (graphe non pondéré)	17
2.3	Matrice d'adjacences (graphe pondéré)	18
2.4	Liste d'adjacence	19, 20
2.5	Choix d'un système de représentation	21
2.6	Recours à des types spécifiques (POO, matrices creuses)	22
2.7	Notions mentionnées mais non expliquées (chapitre 2)	—
2.8	Fiche de synthèse du chapitre 2	—
2.9	Exercices originaux avec corrections	—

2.1 Page 16 : séparation

La page 16 est une page de séparation intitulée « Représentation informatique ». Elle ne contient aucune notion exploitable.

2.2 Matrice d'adjacences — graphe non pondéré (p. 17)

Définition — matrice d'adjacences

On numérote (ou on nomme) les sommets, et on construit un tableau carré M de |S| lignes et |S| colonnes. La case M[u][v] contient **1** s'il existe une arête entre u et v, et **0** sinon. Pour un graphe orienté, M[u][v] = 1 s'il existe un arc **de u vers v**.

Intuition — à quoi ça sert

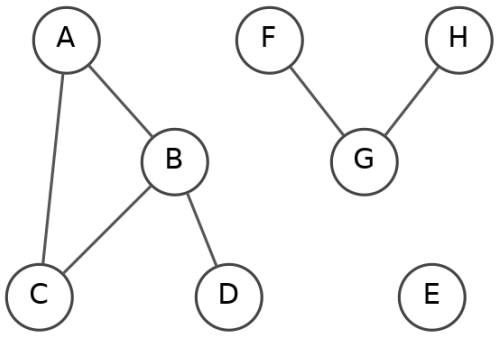
C'est un « tableau de croisement » comme une grille de matchs : la ligne indique d'où l'on part, la colonne où l'on arrive. Pour savoir si deux sommets sont reliés, on regarde **une seule case**, sans rien parcourir. En contrepartie, le tableau a toujours $|S|^2$ cases, même si le graphe a très peu d'arêtes.

Notation

- M[u][v] : case de la ligne u et de la colonne v.
- Les lignes et colonnes sont repérées par les étiquettes des sommets (A à H) ; dans un programme on les convertit en indices 0, 1, …, $|S| - 1$.

Exemple du cours (p. 17)

Le graphe de la p. 17 est non orienté, non pondéré, à 8 sommets. Ses arêtes sont A–B, A–C, B–C, B–D, F–G et G–H ; E est isolé.



	A	B	C	D	E	F	G	H
A	0	1	1	0	0	0	0	0
B	1	0	1	1	0	0	0	0
C	1	1	0	0	0	0	0	0
D	0	1	0	0	0	0	0	0
E	0	0	0	0	0	0	0	0
F	0	0	0	0	0	0	1	0
G	0	0	0	0	0	1	0	1
H	0	0	0	0	0	0	1	0

Figure 2.1 — Graphe de la p. 17 et sa matrice d'adjacences (identique à celle de la p. 17).

Propriété — lecture d'une matrice d'adjacences (non orienté, sans boucle)

- **Symétrie** : M[u][v] = M[v][u], car une arête {u, v} est la même dans les deux sens.
- **Diagonale nulle** : M[u][u] = 0, puisqu'il n'y a pas de boucle ; une boucle sur u se noterait M[u][u] = 1.
- **Somme d'une ligne** = degré du sommet. Ici : A = 2, B = 3, C = 2, D = 1, E = 0, F = 1, G = 2, H = 1.
- **Somme de toutes les cases** = 2|A| (chaque arête est comptée deux fois). Ici 12 = 2 × 6 ✓.
- Une **ligne de zéros** signale un sommet isolé (ligne E).

Propriété — graphe orienté

La matrice n'est en général **pas symétrique**. La somme de la ligne u donne le **degré sortant** de u ; la somme de la colonne v donne le **degré entrant** de v. Les « arêtes symétriques » de la p. 9 (A → B et B → A) donnent deux cases à 1 en position symétrique.

« Il est également possible de travailler avec des booléens »

Les cases ne valent que 0 ou 1 : on peut donc les stocker comme des **booléens** (Vrai/Faux), voire des bits isolés, ce qui réduit la mémoire d'un facteur 8 par rapport à un octet par case. Les opérations logiques (« et », « ou ») deviennent alors possibles directement sur les lignes.

Algorithmes élémentaires sur une matrice

```
FONCTION arete_existe(matrice M, sommet u, sommet v)
DÉBUT
    RETOURNER M[u][v] ≠ 0                 // coût O(1)
FIN

PROCÉDURE parcourir_voisins(matrice M, sommet u)
DÉBUT
    POUR chaque sommet v de G FAIRE       // on doit balayer toute la ligne
        SI M[u][v] ≠ 0 ALORS
            traiter v                     // coût O(|S|) même si u a peu de voisins
FIN

FONCTION construire_matrice(graphe G)
DÉBUT
    M ← matrice |S| × |S| remplie de 0    // coût O(|S|²)
    POUR chaque arête {u, v} de G.A FAIRE
        M[u][v] ← 1
        M[v][u] ← 1                       // omis si le graphe est orienté
    RETOURNER M                           // coût total O(|S|² + |A|)
FIN
```

Exemple — supplémentaire : calcul sur la matrice

Les calculs mathématiques sont un point fort de la matrice (p. 21 : « calculs mathématiques sur le graphe »). Le produit matriciel M × M a pour case (u, v) le **nombre de façons d'aller de u à v en exactement 2 pas**. Sur la figure 2.1 : $(M^2)[A][B] = 1$ (par C), $(M^2)[A][D] = 1$ (par B), et $(M^2)[B][B] = 3$, car on peut aller de B vers chacun de ses 3 voisins et revenir : la diagonale de M^2 donne les **degrés**. Plus généralement, $(M^k)[u][v]$ compte les parcours de longueur k (hors PDF).

Attention — erreurs fréquentes

- La symétrie n'est vraie que pour un graphe **non orienté**.
- Pour un graphe non orienté, M[u][v] = 1 se lit « arête », pas « arc » : ne pas compter chaque arête une seule fois en sommant toute la matrice (on obtient 2|A|).
- La taille est $|S|^2$ **indépendamment** du nombre d'arêtes : un graphe vide à 1000 sommets occupe 1 000 000 de cases.

Exemple — supplémentaire : cas limites

- **Graphe sans arête** à 4 sommets : matrice 4 × 4 entièrement nulle.
- **Clique K_4** : des 1 partout sauf la diagonale (12 cases à 1 = 2 × 6 arêtes).
- **Graphe orienté P → Q → P** (arêtes symétriques) : M[P][Q] = M[Q][P] = 1 ; avec seulement P → Q, M[P][Q] = 1 et M[Q][P] = 0.

2.3 Matrice d'adjacences — graphe pondéré (p. 18)

Définition — matrice des poids

Dans un graphe **pondéré**, la case M[u][v] contient le **poids** w(u, v) de l'arête {u, v} au lieu de 1. Les paires non reliées contiennent 0.

Attention — « On présuppose ici qu'aucune arête n'est de poids nul »

Le 0 sert à dire « pas d'arête ». Si une arête réelle avait un poids nul, on ne pourrait plus la distinguer d'une absence d'arête. C'est la raison de la précaution du cours. **Complément** : quand les poids peuvent être nuls (ou négatifs, comme à partir de la p. 107), on marque l'absence d'arête par une valeur spéciale (∞ pour les plus courts chemins, ou « vide »), et non par 0.

Exemple du cours (p. 18)

Le graphe de la p. 18 est non orienté et pondéré : 9 sommets, 11 arêtes. Ses deux composantes connexes sont {A, B, C, D, E, F} et {G, H, I}.

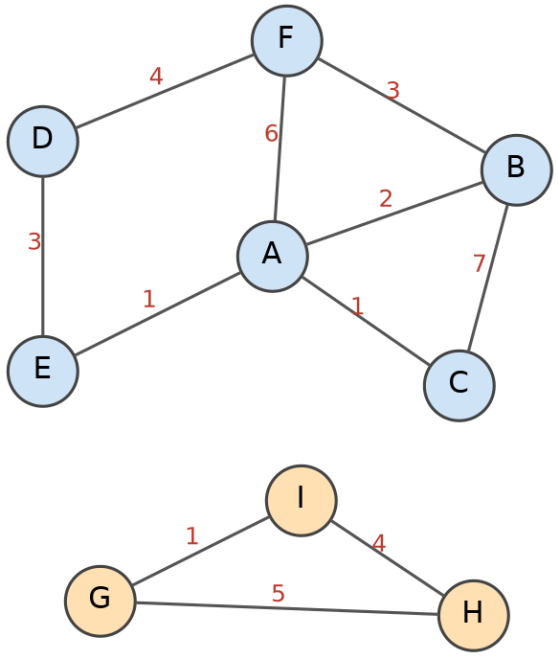


Figure 2.2 — Graphe pondéré de la p. 18 (poids en rouge). Composantes connexes en bleu et en orange.

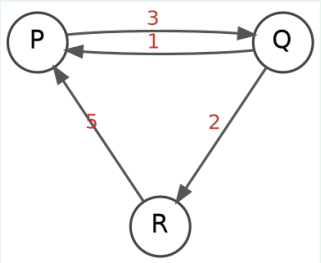
	A	B	C	D	E	F	G	H	I
A	0	2	1	0	1	6	0	0	0
B	2	0	7	0	0	3	0	0	0
C	1	7	0	0	0	0	0	0	0
D	0	0	0	0	3	4	0	0	0
E	1	0	0	3	0	0	0	0	0
F	6	3	0	4	0	0	0	0	0
G	0	0	0	0	0	0	0	5	1
H	0	0	0	0	0	0	5	0	4
I	0	0	0	0	0	0	1	4	0

Figure 2.3 — Matrice d'adjacences pondérée du graphe de la figure 2.2 (p. 18). Elle est symétrique ; les arêtes A–B (2) et B–A (2) sont deux cases.

Propriété

- Même symétrie, même diagonale nulle qu'en non pondéré.
- 22 cases non nulles = 2 × 11 arêtes ; la matrice a 9 × 9 = 81 cases, donc 59 cases « inutiles » (zéros). La proportion de zéros est ce qui motivera les matrices creuses (p. 22).

Exemple — supplémentaire : graphe orienté pondéré



	P	Q	R
P	0	3	0
Q	1	0	2
R	5	0	0

Arcs P → Q (3), Q → P (1), Q → R (2), R → P (5). La matrice n'est **pas symétrique** : M[P][Q] = 3 mais M[Q][P] = 1 ; on peut même avoir un arc sans arc inverse (Q → R existe, pas R → Q).

2.4 Liste d'adjacence (pp. 19, 20)

Définition — liste d'adjacence

On associe à **chaque sommet la liste de ses voisins** (pour un graphe orienté : la liste de ses **successeurs**). On ne stocke donc que les arêtes qui existent.

Intuition

C'est l'annuaire d'un réseau social : pour chaque personne, la liste de ses contacts. Pour savoir à qui parler, on lit la liste ; on ne parcourt pas la liste de toutes les personnes de la planète comme avec la matrice.

Exemple du cours (p. 19)

Même graphe que la p. 17. La p. 19 écrit l'association sous forme d'un « dictionnaire » (clé : sommet, valeur : liste de voisins) :

```
{
  "A": [ "B", "C" ],
  "B": [ "A", "C", "D" ],
  "C": [ "A", "B" ],
  "D": [ "B" ],
  "E": [ ],
  "F": [ "G" ],
  "G": [ "F", "H" ],
  "H": [ "G" ]
}
```

Propriété

- Chaque arête {u, v} apparaît **deux fois** (v dans la liste de u, u dans celle de v) ; chaque arc (u, v) une seule fois. Le nombre total d'entrées dans les listes est donc 2|A| (non orienté) ou |A| (orienté). Ici : 2 + 3 + 2 + 1 + 0 + 1 + 2 + 1 = 12 = 2 × 6 ✓.
- **La longueur de la liste de u est le degré de u**.
- Un sommet isolé (E) **doit quand même figurer** comme clé, avec une liste vide : sinon il disparaîtrait du graphe.
- Mémoire : |S| listes plus 2|A| entrées, soit un coût en **O(|S| + |A|)** : proportionnel à la taille réelle du graphe.

Choix d'implémentation (p. 20)

Le cours souligne que « l'implémentation de l'association et des listes reste à déterminer ». La p. 20 énumère les options ; voici ce que chacune signifie.

Choix (p. 20)	Options	Explication et conséquence
Stockage de l'ensemble du graphe	table de hachage ; tableau indicé sur des entiers	**Table de hachage** : accès à la liste d'un sommet à partir de son nom (« A », « B »...) en temps moyen constant. **Tableau indicé** : les sommets sont numérotés 0, 1, ..., |S| − 1 et l'on accède directement à la case u (le plus rapide, mais il faut numéroter les sommets).
Stockage de chaque liste	tableau de sommets ; liste chaînée	**Tableau** : accès direct à la case k, contigu en mémoire, ajout plus coûteux s'il faut agrandir. **Liste chaînée** : chaque élément pointe vers le suivant ; ajout en tête en O(1), mais pas d'accès direct à la case k.
Trier ou non les listes	recherche dichotomique possible	Si la liste est triée (et stockée en tableau), tester « v est-il voisin de u ? » coûte O(log d(u)) au lieu de O(d(u)). Inconvénient : maintenir l'ordre à chaque ajout.

Choix (p. 20)	Options	Explication et conséquence
Contenu de la liste	labels ; pointeurs vers sommets ; pointeurs vers objets arêtes	Étiquettes : simple, mais il faut retrouver le sommet ensuite. Pointeurs vers sommets : accès immédiat à l'objet voisin. Pointeurs vers arêtes : permet de stocker un poids, une étiquette ou une capacité sur l'arête elle-même.

Attention — dichotomie et liste chaînée

La recherche dichotomique exige un **accès direct** à la case du milieu : elle fonctionne sur un tableau trié, pas sur une liste chaînée (où l'on arriverait au milieu en parcourant la moitié des éléments). **Complément** : c'est une conséquence de la structure de données, pas une phrase du cours.

Extension aux poids et étiquettes (p. 19)

« On peut par ailleurs étendre le système pour inclure des poids ou des étiquettes » : on stocke dans la liste des **couples (voisin, poids)**. Exemple pour le graphe pondéré de la p. 18 :

```
A : (B,2) (C,1) (E,1) (F,6)       D : (E,3) (F,4)       G : (H,5) (I,1)
B : (A,2) (C,7) (F,3)             E : (A,1) (D,3)       H : (G,5) (I,4)
C : (A,1) (B,7)                   F : (A,6) (B,3) (D,4) I : (G,1) (H,4)
```

Algorithmes élémentaires sur les listes

```
FONCTION arete_existe(listes L, sommet u, sommet v)
DÉBUT
    POUR chaque t dans L[u] FAIRE
        SI t = v ALORS RETOURNER Vrai
    RETOURNER Faux                        // coût O(d(u)), ou O(log d(u)) si L[u] est trié
FIN

PROCÉDURE parcourir_voisins(listes L, sommet u)
DÉBUT
    POUR chaque t dans L[u] FAIRE
        traiter t                         // coût O(d(u)) : uniquement les vrais voisins
FIN

FONCTION construire_listes(graphe G)
DÉBUT
    POUR chaque sommet s de G.S FAIRE
        L[s] ← liste vide
    POUR chaque arête {u, v} de G.A FAIRE
        ajouter v à L[u]
        ajouter u à L[v]                  // omis si le graphe est orienté
    RETOURNER L                           // coût total O(|S| + |A|)
FIN
```

Exemple — supplémentaire : graphe orienté

Graphe orienté à 4 sommets : 1 → 2, 2 → 3, 3 → 1, 3 → 4, 4 → 4 (figure 2.4, exercice A). Listes de successeurs : 1 : [2] ; 2 : [3] ; 3 : [1, 4] ; 4 : [4]. Le sommet 4 figure dans sa propre liste (boucle). **Piège** : retrouver les **prédécesseurs** de 4 (ici 3 et 4) oblige à parcourir toutes les listes (coût O(|S| + |A|)) ; on peut stocker en plus une liste des prédécesseurs si c'est une opération fréquente (hors PDF).

Attention — conversions et oublis

- Convertir une **matrice en listes** coûte $O(|S|^2)$ (il faut lire toute la matrice) ; construire les listes à partir de l'ensemble des arêtes coûte O(|S| + |A|).
- Oublier de symétriser (ajouter v à L[u] **et** u à L[v]) donne en réalité un graphe **orienté**.

2.5 Choix d'un système de représentation (p. 21)

La p. 21 ne tranche pas : elle liste les **critères** à examiner avant de choisir. La conclusion est que le choix dépend du graphe et de ce que l'on veut en faire.

Critère (p. 21)	Question à se poser	Penche vers…
Occupation mémoire : taille du graphe (« ordre »)	Combien de sommets $\|S\|$? Un tableau $\|S\|^2$ est-il supportable ?	listes si $\|S\|$ est grand
Occupation mémoire : densité	Le graphe a-t-il peu d'arêtes ($\|A\|$ proche de $\|S\|$) ou beaucoup ($\|A\|$ proche de $\|S\|^2$) ?	listes si peu dense ; matrice si dense
Test d'existence entre deux sommets quelconques	Faut-il décider très souvent si u–v existe ?	matrice (O(1))
Itération sur le voisinage d'un sommet	Faut-il lister les voisins de u ?	listes (O(d(u)) au lieu de O($\|S\|$))
Exploration systématique du graphe	Parcours en profondeur / largeur (chapitre 3)	listes : coût O($\|S\| + \|A\|$)
Calculs mathématiques sur le graphe	Puissances de matrice, algèbre linéaire	matrice
Partage du graphe avec d'autres utilisateurs	Envoi, sauvegarde, échange de données	listes ou liste d'arêtes : texte court et lisible
Programmation orientée objet	Étiquettes, poids, objets arêtes	listes (la POO les favorise, p. 21)

Comparaison synthétique (complément)

Opération	Matrice d'adjacences	Liste d'adjacence (non triée)
Mémoire	$\|S\|^2$ cases	$\|S\| + 2\|A\|$ entrées (non orienté)
Test « u–v existe ? »	O(1)	O(d(u))
Énumérer les voisins de u	O($\|S\|$)	O(d(u))
Parcourir toutes les arêtes	O($\|S\|^2$)	O($\|S\| + \|A\|$)
Ajouter une arête	O(1)	O(1) en tête de liste
Supprimer une arête	O(1)	O(d(u) + d(v))
Ajouter un sommet	recréer le tableau, O($\|S\|^2$)	O(1)
Calcul matriciel	naturel	peu naturel

Complément

Ce tableau est ma synthèse des critères de la p. 21 (non présent dans le PDF) ; les coûts sont ceux d'implémentations standard (tableau statique pour la matrice, listes chaînées pour les listes).

Exemple — supplémentaire : réseau routier contre petit graphe dense

- **Réseau routier** : $|S| = 100\,000$ carrefours, $|A| = 300\,000$ routes (chaque carrefour a en moyenne 6 routes). La matrice a $100\,000^2 = \mathbf{10^{10}}$ cases. Les listes contiennent $|S| + 2|A| = \mathbf{700\,000}$ entrées : environ 14 000 fois moins. Le choix est évident.
- **Petite clique** : $|S| = 10$, $|A| = 45$ (K_{10}). Matrice : 100 cases ; listes : 10 + 90 = 100 entrées, mais chaque entrée est plus lourde (un pointeur) et le test d'existence est plus lent. La matrice gagne.

2.6 Recours à des types spécifiques (p. 22)

Programmation orientée objet (POO)

La p. 22 explique qu'en POO on stocke facilement les **labels, étiquettes et poids**, que la POO est « particulièrement adaptée aux listes d'adjacence », et qu'on peut coder **séparément les**

sommets et les arêtes : la liste d'adjacence est alors une **liste d'arêtes** (objets).

```
STRUCTURE Sommet
    etiquette                   // le nom (A, B, ...)
    aretes                      // liste des objets Arete incidents à ce sommet

STRUCTURE Arete
    extremite1, extremite2      // les deux sommets reliés
    poids                       // optionnel
    etiquette                   // optionnelle
```

Intuition

Une arête devient un **objet à part entière** : on peut lui attacher un poids, une capacité, une couleur, sans changer la structure. C'est ce que permet le dernier choix de la p. 20 (« la liste pointe vers des objets représentant les arêtes »).

Matrices creuses

Une matrice est dite **creuse** quand la grande majorité de ses cases valent 0. La p. 22 précise que certaines bibliothèques spécialisées proposent un **type « matrice creuse »** : on ne stocke que les cases non nulles, avec un « choix fin entre plusieurs algorithmes de stockage » ; c'est adapté aux graphes **peu ou moyennement denses**, et optimise le coût en mémoire et en temps.

Exemple — supplémentaire : idée du stockage creux

Pour la matrice de la p. 17, au lieu de 64 cases, on peut mémoriser uniquement les 12 cases non nulles sous forme de triplets (ligne, colonne, valeur) : (A,B,1), (A,C,1), (B,A,1), (B,C,1), (B,D,1), (C,A,1), (C,B,1), (D,B,1), (F,G,1), (G,F,1), (G,H,1), (H,G,1). On retrouve, déguisée, la liste d'adjacence.

Complément : les variantes d'organisation de ces triplets (par ligne, par colonne, par coordonnées) sont les « algorithmes de stockage » évoqués par la diapositive ; les détails dépendent de la bibliothèque et ne sont pas dans le PDF.

2.7 Notions mentionnées mais non expliquées dans le PDF (chapitre 2)

Notions citées, supposées connues ou seulement évoquées dans les pages 16 à 22.

Lettre	Notion	Page(s)	Statut dans le PDF
L	Matrice, symétrie d'une matrice, tableau à deux dimensions	17, 18	supposée connue
M	Booléen	17	cité
N	Table de hachage	20	citée
O	Tableau indicé sur des entiers	20	cité
P	Liste chaînée, pointeur, référence	20	cités
Q	Recherche dichotomique	20	citée
R	Ordre du graphe	21	cité entre guillemets
S	Programmation orientée objet (objet, attribut)	21, 22	supposée connue
T	Matrice creuse et formats de stockage	22	citée sans détail
U	Calculs mathématiques sur le graphe	21	évoqués
V	Partage du graphe avec d'autres utilisateurs (sérialisation)	21	évoqué

L. Matrice et symétrie

Définition

Une **matrice** est un tableau rectangulaire de nombres rangés en lignes et colonnes ; M[i][j] est la case de la ligne i et de la colonne j. Elle est **symétrique** si M[i][j] = M[j][i] pour tous i, j : le tableau est inchangé quand on l'échange par rapport à sa diagonale.

M. Booléen

Définition

Une valeur qui ne peut prendre que deux états : Vrai (1) ou Faux (0). Occupe théoriquement 1 bit.

N. Table de hachage

Définition

Structure qui associe une **valeur** à une **clé** (ici : un nom de sommet → sa liste de voisins). Une **fonction de hachage** transforme la clé en un numéro de case ; on lit ou écrit directement cette case. Deux clés peuvent donner le même numéro (**collision**) : on gère le cas (par exemple en chaînant les éléments en conflit). Accès en temps moyen constant.

Exemple

Clé « C » → numéro de case 2 (par exemple) → on y trouve la liste [A, B]. On n'a pas parcouru les autres sommets.

O. Tableau indicé sur des entiers

Définition

Un tableau dont l'indice est un entier de 0 à n − 1. Si les sommets sont numérotés ainsi, la liste d'adjacence est simplement un tableau de listes, et la matrice un tableau de tableaux. Pour des sommets nommés, on garde une table de correspondance nom ↔ numéro.

P. Liste chaînée, pointeur

Définition

Un **pointeur** (ou **référence**) est une valeur qui désigne l'emplacement d'un autre objet en mémoire. Une **liste chaînée** est une suite d'éléments dont chacun contient une donnée et un pointeur vers l'élément suivant. Insertion en tête en O(1) ; accès au k-ième élément en O(k).

Q. Recherche dichotomique

Définition

Recherche dans un **tableau trié** : on compare à l'élément du milieu, on garde la moitié qui peut contenir la valeur, et on recommence. Coût en O(log n).

Exemple

Dans la liste triée [B, D, F, H, K, M, P] on cherche K : milieu = H (K est plus grand, on garde [K, M, P]) ; milieu = M (K est plus petit, on garde [K]) ; trouvé : 3 comparaisons au lieu de 5.

R. Ordre du graphe

Cohérent avec la section 1.10 : l'**ordre** d'un graphe est son nombre de sommets |S| (la **taille** étant |A|). La p. 21 s'en sert comme critère mémoire.

S. Programmation orientée objet

Style de programmation où les données et les opérations qui les manipulent sont regroupées dans des **objets**. Un objet a des **attributs** (ici : étiquette, poids) et est créé à partir d'une **classe** (ici : Sommet, Arete). Les attributs de sommets du chapitre 3 (« découvert », « distance »…) sont exactement ce genre de champs.

T. Matrice creuse

Voir la section 2.6. Mesure : la matrice de la p. 18 contient 59 zéros sur 81 cases (≈ 73 %), celle de la p. 17 en contient 52 sur 64 (≈ 81 %), et la proportion augmente avec la taille d'un réseau peu dense.

U. Calculs mathématiques sur le graphe

Il s'agit des calculs d'algèbre linéaire sur la matrice d'adjacences (voir l'exemple de M^2 en section 2.2) : nombre de parcours d'une longueur donnée, détection de composantes, etc. Le détail n'est pas dans le cours.

V. Partage du graphe

Pour transmettre un graphe à un autre programme ou une autre personne, on l'**écrit dans un format texte** (sérialisation). Le dictionnaire de la p. 19 en est un exemple direct. Une matrice 100 000 × 100 000 serait impraticable à partager ; une liste d'arêtes (une ligne « u v » par arête) est compacte.

2.8 Fiche de synthèse du chapitre 2

Notion	À retenir	Page
Matrice d'adjacences	tableau |S| × |S| ; M[u][v] = 1 (ou poids) si arête, 0 sinon ; symétrique si non orienté	17, 18
Lectures de la matrice	somme de ligne = degré (sortant en orienté) ; somme de colonne = degré entrant ; total = 2|A| en non orienté	17
Poids nul	interdit dans la matrice pondérée (0 = pas d'arête)	18
Liste d'adjacence	à chaque sommet, la liste de ses voisins ; chaque arête apparaît 2 fois en non orienté ; mémoire O(|S| + |A|)	19

Notion	À retenir	Page
Choix d'implémentation	table de hachage ou tableau pour l'ensemble ; tableau ou liste chaînée pour chaque liste ; trié ou non ; contenu : labels, pointeurs vers sommets ou arêtes	20
Critères de choix	mémoire (ordre, densité), test d'arête, voisinage, exploration, calculs, partage	21
Compromis	matrice : test O(1) mais $\|S\|^2$ cases ; listes : O($\|S\| + \|A\|$) mais test O(d(u))	21
POO	arêtes et sommets comme objets ; favorise les listes d'adjacence	21, 22
Matrice creuse	ne stocke que les cases non nulles ; pour graphes peu ou moyennement denses	22

2.9 Exercices originaux avec corrections

Exercice A — graphe orienté

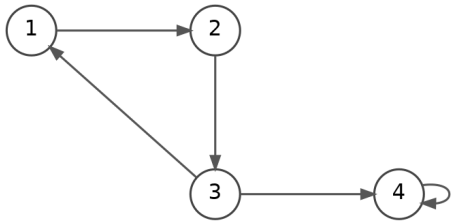


Figure 2.4 — Graphe orienté de l'exercice A : arcs 1→2, 2→3, 3→1, 3→4, 4→4.

Exercice

(a) Écrire la matrice d'adjacences. (b) Écrire les listes de successeurs. (c) Calculer les degrés entrants et sortants. (d) La matrice est-elle symétrique ? (e) Comparer l'occupation mémoire.

Solution

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	1	0	0	1
4	0	0	0	1

(a) Ligne = origine, colonne = destination. La case (4, 4) = 1 représente la boucle.

(b) 1 : [2] ; 2 : [3] ; 3 : [1, 4] ; 4 : [4].

(c) Degrés sortants (sommes de lignes) : 1, 1, 2, 1 (total 5 = |A|). Degrés entrants (sommes de colonnes) : 1, 1, 1, 2 (total 5).

(d) Non symétrique : M[1][2] = 1 mais M[2][1] = 0. (La symétrie est propre aux graphes non orientés.)

(e) Matrice : 16 cases. Listes : 4 + 5 = 9 entrées.

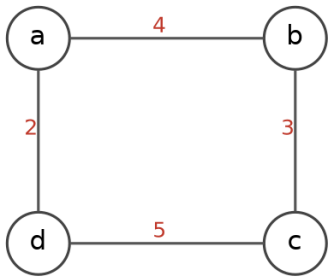
Exercice B — de la matrice au graphe

Exercice

Voici la matrice d'adjacences pondérée d'un graphe non orienté. Dessiner le graphe et écrire ses listes d'adjacence avec poids.

	a	b	c	d
a	0	4	0	2
b	4	0	3	0
c	0	3	0	5
d	2	0	5	0

Solution



La matrice est symétrique, diagonale nulle : graphe non orienté sans boucle. Lecture ligne par ligne : a–b (4), a–d (2), b–c (3), c–d (5). C'est un **cycle à 4 sommets**, tous de degré 2.

Listes : a : (b,4) (d,2) ; b : (a,4) (c,3) ; c : (b,3) (d,5) ; d : (a,2) (c,5). Somme de toutes les cases = 28 = 2 × 14, où 14 = 4 + 3 + 5 + 2 est la somme des poids des arêtes.

Exercice C — quelle représentation choisir ?

Exercice

Un graphe a 1000 sommets et 5000 arêtes (non orienté). (a) Calculer sa densité. (b) Comparer le nombre de cases d'une matrice et le nombre d'entrées de listes. (c) Avec les hypothèses « 1 bit par case de matrice » et « 4 octets par entrée de liste », laquelle occupe le moins de mémoire ? (d) À partir de quelle densité environ la matrice devient-elle plus économe ?

Solution

(a) Densité = 2 × 5000 / (1000 × 999) = 10 000 / 999 000 ≈ **1 %**.

(b) Matrice : 1000^2 = 1 000 000 cases. Listes : 1000 + 2 × 5000 = 11 000 entrées, soit environ 91 fois moins d'éléments.

(c) Matrice : 10^6 bits = 125 000 octets. Listes : 11 000 × 4 = 44 000 octets. **Les listes** gagnent.

(d) Pour |S| grand, listes ≈ 4 × 2|A| = 8|A| octets, matrice = $|S|^2$ / 8 octets. Égalité quand |A| = $|S|^2$ / 64, soit une densité 2|A| / $|S|^2$ ≈ 1/32 ≈ **3 %**. Au-delà, la matrice est plus compacte. (Résultat **hypothétique** : il dépend entièrement du coût supposé d'une case et d'une entrée.)

Exercice D — modifier le graphe

Exercice

Sur le graphe de la p. 17, on ajoute l'arête D–E. Quelles cases de la matrice changent ? Que devient la liste d'adjacence ? Qu'en est-il si le graphe est orienté et que l'on ajoute seulement l'arc D → E ?

Solution

Non orienté : M[D][E] et M[E][D] passent à 1 (deux cases symétriques) ; on ajoute E à la liste de D et D à la liste de E (qui n'est plus vide). Orienté : seule M[D][E] passe à 1 ; on ajoute E uniquement à la liste de D ; la liste de E reste vide.

Contrôle : en non orienté, la somme de la matrice passe de 12 à 14 = 2 × 7 arêtes ; en orienté, de 12 à 13.